

SYSTEMVERILOG / UVM

시각 학습 노트

1~10단계 · 핵심 개념과 데이터 흐름

LANGUAGE

class · process · constraint · interface

VERIFICATION

transaction · component · checking · coverage

2026.10.10 · VISUAL SUMMARY · 상세 설명은 MD 학습 노트

개념·코드 해석용 자료 / UVM simulator 미실행

LEARNING STAGE



1단계

객체지향 문법

class · object · handle · inheritance · polymorphism

객체지향 문법 전체 지도

UVM의 transaction과 component를 읽기 위한 기초

1

설계와 생성

class · object · new()

2

참조와 복사

handle · shallow/deep

3

확장

inheritance · overriding

4

동적 선택

virtual · polymorphism

5

추상화

abstract · pure virtual

6

재사용과 범위

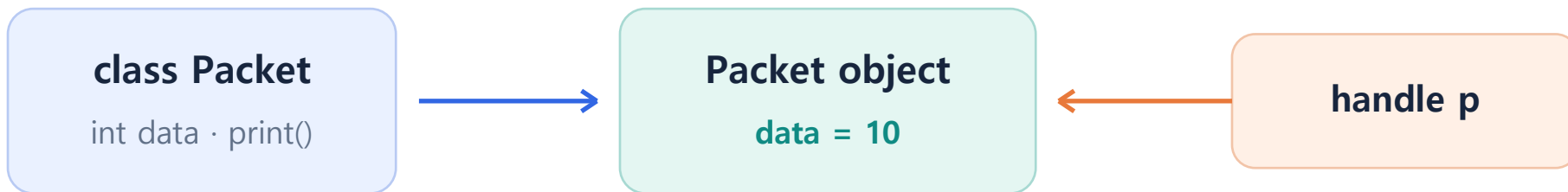
parameter · static · 접근 제한

UVM 연결 transaction은 object이며 handle로 이동한다.

factory와 polymorphism은 부모 type으로 여러 자식 transaction/component를 교체 가능하게 만든다.

설계도, 실체, 그리고 handle

선언과 생성을 분리해서 생각한다



Packet p; handle만 선언 · p == null

p = new(); object 생성 · p가 object를 가리킴

손으로 상태를 그려보기

```

Packet a, b;
a = new();
a.data = 7;
b = a;
a = new();
a.data = 30;
  
```

마지막 a와 b가 가리키는 object · data 값 · object 수를 표시

같은 값인가, 같은 object인가

복사의 핵심은 중첩 handle 공유 여부

SHALLOW COPY



Packet 2개 · Header 1개 · b.hdr.id 변경은 a에서도 보임

DEEP COPY

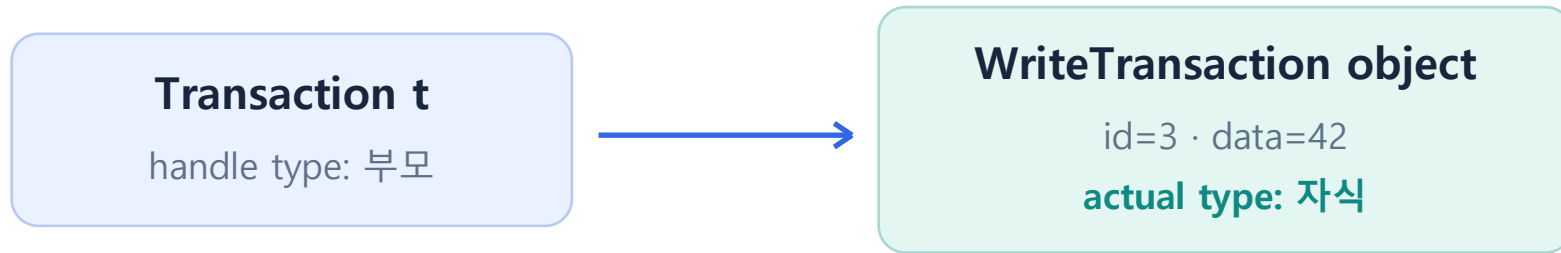


Packet도 Header도 각각 독립

판별 b.hdr.id = a.hdr.id → int 값 복사 b.hdr = a.hdr → handle 대입 · 공유

handle type과 actual object type을 분리하라

virtual은 실제 object에 맞는 구현을 선택한다



virtual 없음 handle type 기준
t.print() → base

virtual 있음 actual object type 기준
t.print() → write

말로 설명하기 "부모 handle 하나로 여러 자식 object의 동작을 선택할 수 있다."

구현을 요구하고, type을 바꿔 재사용한다

ABSTRACT CLASS

virtual class Transaction

pure virtual function
void print();



자식 print()
구현 필수

부모 object 생성 × 부모 handle 선언 ○

PARAMETERIZED CLASS

class Box #(type T = int);

Box#(int)

Box#(string)

서로 다른 class type · int는 기본 type

OVERRIDING 자식이 부모와 같은 method의 자기 구현을 제공한다.

부모 구현 자체는 바뀌지 않는다. SystemVerilog는 일반적인 method overloading을 지원하지 않는다.

핵심 추상 class는 공통 계약을 만들고, parameterized class는 공통 구조를 여러 type에 재사용한다.

누구의 값이며, 어디까지 보이는가

instance/static과 접근 제한자를 한 장에서 비교

INSTANCE

a.id=1 b.id=2

object마다 별도 저장

STATIC

Packet::count = 2

class가 하나를 공유 · object
없이 static method 호출

접근 가능 범위

선언	같은 class	자식 class	class 밖
int data	○	○	○
protected int data	○	○	×
local int data	○	×	×

Constructor handle 선언 → count 변화 없음 new() 실행 → constructor 안의 count++ 실행

현재 object와 부모 구현을 구분한다

직접 그리며 1단계를 점검한다

this.data = data; 현재 object property ← argument **super.new(id);** 부모 constructor 호출

그리기 문제

```
Transaction t;  
WriteTransaction w;  
w = new(3, 42);  
t = w;
```

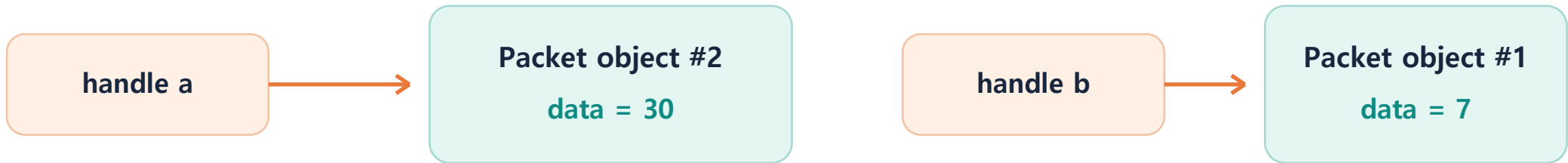
handle type · actual object type · object 수 · id/data를 표시

말로 답하기 ① t = w가 object를 만드는가? ② virtual print()이면 어느 구현? ③ shallow copy의 공유 지점?

그리기 문제 답안

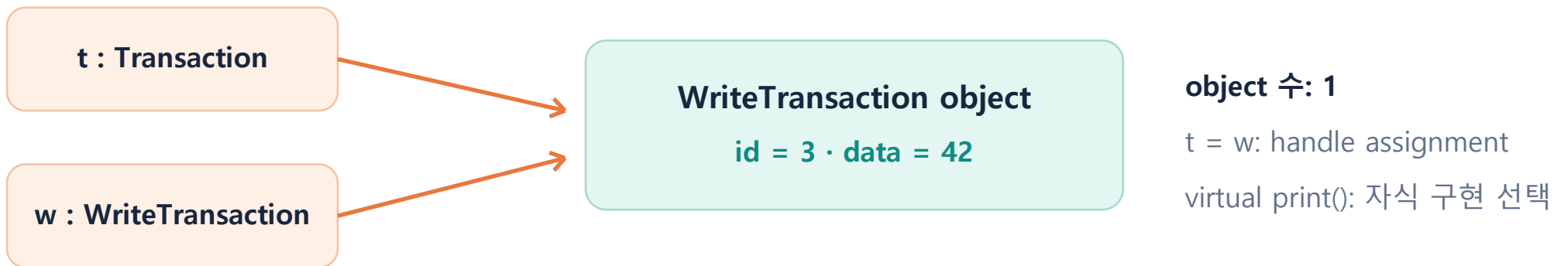
먼저 직접 그린 뒤 비교한다

설계도·실체·handle 문제 답안



object는 총 2개. $b = a$ 에서는 공유했다.
이후 $a = \text{new}()$ 가 a 의 연결 대상만 바꾼다.

this·super / 상속 handle 문제 답안



정답을 외우기보다 “new() 횟수 → handle 연결 → actual object type” 순서로 그린다.

LEARNING STAGE



2단계

데이터 구조와 Process 통신

array · queue · process · event · semaphore · mailbox

데이터를 담고, process를 연결한다

자료구조 선택과 thread 동기화의 기준을 세운다

1

배열 배치

packed / unpacked

2

크기 관리

fixed / dynamic

3

가변 저장

queue / associative

4

의미 있는 type

enum / struct / typedef

5

동시 실행

fork / join 계열

6

동기화

event / semaphore / mailbox

선택 질문 크기가 고정인가? 순서인가 ID인가? 데이터를 전달하는가? 자원 접근을 제한하는가?
구조 이름을 외우기보다 해결하려는 문제부터 구분한다.

변수 이름 앞은 packed, 뒤는 unpacked

폭과 element 수를 분리해서 읽는다

logic [7:0] memory [0:15];

[7:0] · PACKED

각 element의 폭 = 8bit



[0:15] · UNPACKED

element 수 = 16개

memory[3] → logic [7:0] element 하나

memory[3][2] → 네 번째 element의 bit 2

주의 int는 element type이다. Array 크기나 element 개수와 혼동하지 않는다.

크기와 접근 방식으로 자료구조를 고른다

FIXED ARRAY

선언 시 크기 고정

int a[4]

연속 index

DYNAMIC ARRAY

new[n]으로 재할당

int a[]

연속 index

QUEUE

push / pop으로 자동 증감

int q[\$]

순서 기반

ASSOCIATIVE

사용한 key만 존재

int a[string]

ID / key 기반

UVM 연결 Queue = in-order pending 목록 · Associative array = transaction ID 기반 out-of-order matching

enum은 값의 집합, struct는 field의 묶음

typedef는 재사용할 type 이름을 만든다

ENUM

IDLE · READ · WRITE · ERROR

숫자 대신 의미 있는 상태 이름

`state.name() → "WRITE"`

STRUCT PACKED

opcode | address | data

관련 field를 하나의 value로 묶음

4 + 8 + 16 = 28bit vector

STRUCT 대입 field 값 복사 · 이후 두 variable은 독립

CLASS 대입 handle 대입 · 같은 object를 공유

구분 enum은 named value type · struct는 복합 value type · typedef는 type의 별칭

fork는 child thread를 만들고, join은 parent의 대기를 결정한다

join

모두 끝날 때까지 대기
남은 child 없음

join_any

하나가 끝나면 진행
나머지는 계속 실행

join_none

즉시 parent 진행
child는 계속 실행

wait fork 모든 child 종료까지 대기
사용 지점 join_none 뒤에 다시 합류

disable fork 현재 process의 활성 자손 종료
사용 지점 join_any 뒤 forever 정리

함정 join_any는 남은 child를 자동으로 종료하지 않는다.

event는 알림, semaphore는 공유 자원 잠금

EVENT

-> **done;** trigger

@done; 다음 trigger 대기

데이터 저장 × · 과거 **trigger** 기억 ×

대기 전에 발생하면 event 유실 가능

SEMAPHORE

get(1) key 없으면 대기

try_get(1) 즉시 1 / 0

put(1) key 반환

put 누락 → 다른 thread deadlock

선택 발생 사실만 알림 → event 공용 bus의 동시 접근 제한 → semaphore

데이터 전달과 기다림을 한 구조에 담는다

producer와 consumer 사이의 synchronized communication



QUEUE와 차이 queue는 저장 구조 · mailbox는 process 사이의 대기과 전달까지 제공

2단계 선택 기준과 오류 패턴

왼쪽 문제를 먼저 가리고 답한다

문제

1. sparse transaction ID 조회
2. producer → consumer 전달
3. 공용 bus 동시 접근 제한
4. reset 완료 사실만 알림
5. join_any 뒤 forever thread 정리

답

1. associative array
2. mailbox
3. semaphore
4. event
5. disable fork

Deadlock 점검 get() 대기 · put() 누락 · event 유실 · forever child · 너무 넓은 disable fork 범위

기억 순서 저장 방식 → index/key → blocking 여부 → 종료 조건

LEARNING STAGE



3단계

Randomization과 Constraint

값 공간 · solver · 분포 · 실패 분석

조건을 만족하는 값을 만들고, 실패를 통제한다

Randomization은 값 생성이 아니라 해를 찾는 과정이다



SOLVER 질문 무엇을 바꿀 수 있는가? 모든 조건의 교집합은 존재하는가? 분포는 의도와 같은가?
실패하면 이전값이 남으므로 반환값을 반드시 확인한다.

rand는 반복 가능, randc는 주기 안에서 비반복

new()는 생성 · randomize()는 값 선택 · 생성만으로 자동 randomization하지 않는다

rand bit [1:0] id;

가능한 예 **0, 0, 2, 1**

같은 값 연속 가능

randc bit [1:0] id;

두 주기 예 **2,0,3,1 | 1,3,0,2**

주기 내부 비반복 · 경계 반복 가능

if (!item.randomize()) 실패 시 0 · field는 이전값 유지

``uvm_fatal("RANDFAIL", "Item randomization failed")`

주의 실패 반환값을 무시하면 stale transaction이 driver로 전달될 수 있다.

초기값 검사가 아니라, 모든 조건을 만족하는 해 탐색

활성 rand field는 미지수이고 비활성 field는 고정값이다

RAND MODE ON

현재 addr = 12

조건 $\text{addr} \in [0:3]$

12를 0~3 중 하나로 변경 → 성공

RAND MODE OFF

고정 addr = 12

조건 $\text{addr} \in [0:3]$

12 $[0:3]$ · 변경 불가 → 실패

constraint 대입문 × randomize가 끝났을 때 반드시 참이어야 하는 식

block 내부 각 식은 기본적으로 AND · solver는 동시에 만족하는 조합을 탐색

판별 순서 바꿀 수 있는 변수 → 활성 조건 → 교집합 → 반환값

inside는 값 공간을 만들고, dist는 상대적 비중을 준다

```
addr inside {[2:5], 9, [12:14]};
```

가능값 $2 \cdot 3 \cdot 4 \cdot 5 \cdot 9 \cdot 12 \cdot 13 \cdot 14$

범위 weight 비교

[0:1] := 10 각 값에 10

[2:5] := 40 각 값에 40

[0:1] :/ 10 전체 10을 분배

[2:5] :/ 40 전체 40을 분배

기억 weight는 횟수 보장이 아니다. 짧은 실행에서 실제 비율은 달라질 수 있다.

한쪽만 제한할지, 모든 경우를 제한할지 선택한다

IMPLICATION

write -> addr inside {[8:15]};

write=1 **addr 8~15**

write=0 이 식은 addr를 제한하지 않음

CONDITIONAL

if (write) addr inside {[8:15]};

else addr inside {[0:3]};

write=1 **addr 8~15**

write=0 **addr 0~3**

논리 해석 $A \rightarrow B \equiv (!A) \text{ OR } B$ A가 거짓이면 implication은 참

선택 조건이 참일 때만 제한 → implication · 두 경우 모두 제한 → if / else

solve before는 분포, soft는 양보 가능한 기본 조건

solve a before b;

a=2 → b 3개

a=3 → b 2개

a=4 → b 1개

a를 먼저 선택: 각 a 약 1/3

soft addr inside {[0:10]};

inline: addr inside {[7:13]};

교집합 7~10 존재

soft 유지 · 최종 7~10

교집합이 없을 때 soft가 양보

공통점 합법 범위와 선택 분포를 구분해서 생각한다.

주의 solve before는 절차적 실행 순서가 아니며, soft는 다른 조건이 생겼다고 즉시 사라지지 않는다.

무엇을 랜덤화하고 어떤 조건을 적용할지 실행 중 조정한다

addr.rand_mode(0)

addr를 현재값으로 고정
관련 **constraint**는 계속 활성

c_addr.constraint_mode(0)

해당 constraint block 비활성
field의 rand mode와는 독립

pre_randomize() mode와 상태 준비

solver 실행 전 rand field 대입값은 다시 변경 가능

post_randomize() 파생값 계산

성공 후 실행 checksum · formatted field

상속 다른 이름 → AND로 추가 같은 이름 → 자식 constraint가 부모 constraint override

3단계 실패 분석 순서

값이 이상할 때 solver 관점으로 역추적한다

1

RAND MODE

현재값을 바꿀 수 있는가?

2

ACTIVE CONSTRAINT

어떤 class/inline 조건이 켜졌는가?

3

SOLUTION SPACE

모든 조건의 교집합이 존재하는가?

4

DISTRIBUTION

dist와 solve before가 의도와 같은가?

5

RETURN VALUE

실패를 보고하고 전달을 멈췄는가?

상태 구분 conflicting = 해 없음 · over-constrained = 합법 상황을 과도하게 제외
under-constrained = 필요한 조건이 빠져 의도하지 않은 값 생성

한 문장 Randomization은 바꿀 수 있는 변수로 모든 활성 조건을 만족하는 해를 찾는 과정이다.

LEARNING STAGE



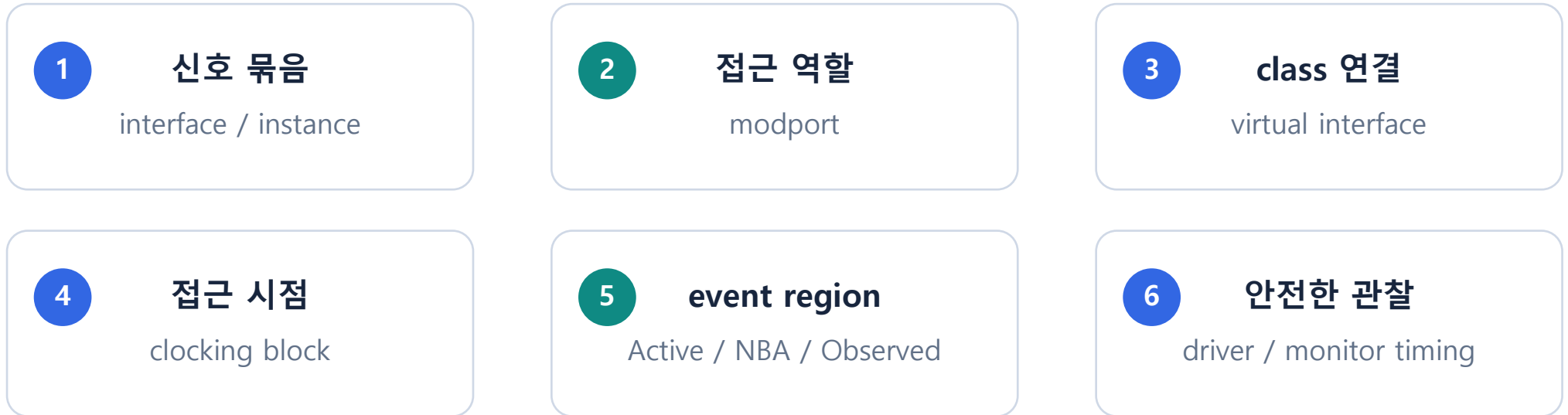
4단계

Interface와 Simulation Timing

interface · modport · virtual interface · clocking block

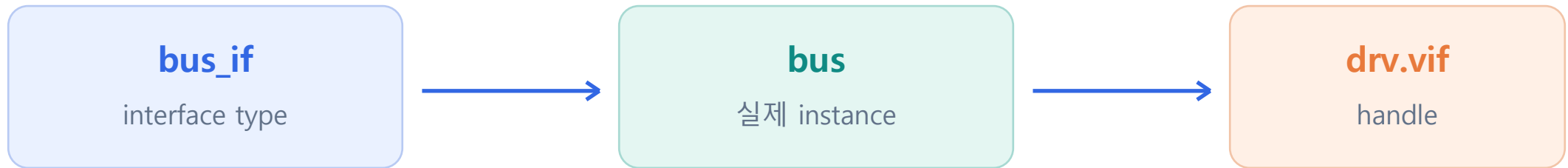
신호를 묶고, 역할과 시점을 분리한다

Interface는 연결 구조이고 clocking block은 timing 계약이다



핵심 질문 실제 신호는 어디에 있는가? 누가 읽고 쓰는가? 어느 event region에서 관찰하는가?
같은 clock edge라는 말만으로는 구동과 sampling 순서가 결정되지 않는다.

type, instance, virtual handle을 분리해서 읽는다



bus_if bus(.clk(clk)); top에서 실제 interface 생성

drv.vif = bus; 복사나 생성이 아니라 handle 연결

clk top의 clock을 interface가 입력으로 받음

signal interface 내부 protocol 신호

modport는 방향, clocking block은 시점

MODPORT DRV

output **valid** · data

input **ready** · clk

누가 무엇을 읽고 쓰는가?

CLOCKING drv_cb

event **@(posedge clk)**

skew **input #1step** · output #0

언제 구동하고 sampling하는가?

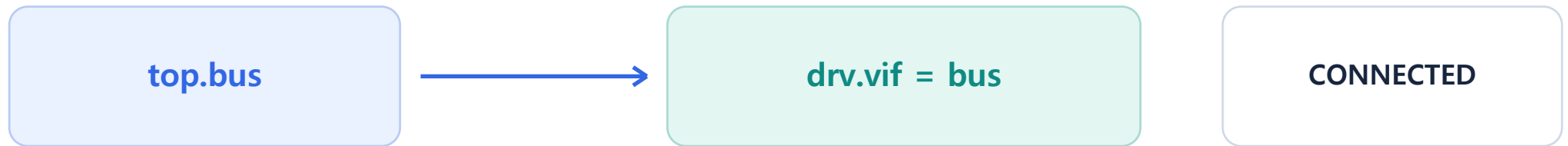
MON 모든 신호 input · 관찰만 수행 · interface 신호를 구동하지 않음

구분 방향 오류는 modport에서, timing race는 clocking block에서 막는다.

class는 실제 interface를 만들지 않고 handle로 가리킨다

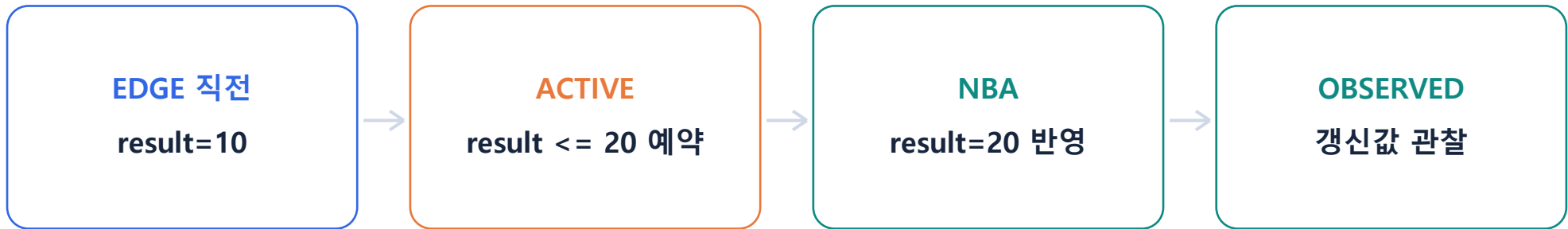


정상 연결



필수 점검 if (vif == null) → fatal 처리 후 signal 접근 중단

같은 posedge라도 update와 관찰 시점은 다르다



blocking = Active에서 즉시 update
위험 **DUT와 TB 실행 순서에 의존**

nonblocking <= NBA에 update 예약
효과 **Active에서는 이전값 유지**

RACE 같은 region의 실행 순서를 가정하면 simulator와 코드 변화에 따라 결과가 달라질 수 있다.

현재 edge에서 구동하고, 다음 edge에서 DUT가 sampling한다

posedge에서 입력을 받는 DUT, ready=1인 예 · clocking output #0 구동



drv_cb output #0 같은 edge에서 DUT와 먼저 실행되려고 경쟁하지 않는다.
다음 sampling edge에 사용할 값을 cycle 사이에 안정적으로 유지한다.

관점 drive edge와 DUT acceptance edge를 구분한다.

같은 edge에서 이전값과 갱신값을 구분해 sampling한다

result: 10

result: 20



EDGE 직전

10



ACTIVE

10



NBA

20



OBSERVED

20

input #1step edge 직전

sampling result 10

input #0 NBA 반영 후

sampling result 20

선택 요청 수락 #1step / 등록 결과 #0 · clocking input #0은 절차문 #0과 다르다.

4단계 연결과 timing 점검 순서

signal 값보다 먼저 instance, 방향, event region을 확인한다

1 INSTANCE

실제 interface가 top에 생성됐는가?

2 VIF

class handle이 실제 instance를 가리키는가?

3 MODPORT

읽기와 쓰기 방향이 역할에 맞는가?

4 CLOCKING

기준 edge와 skew가 명확한가?

5 REGION

Active, NBA, Observed 중 어디에서 보는가?

6 CYCLE

drive와 acceptance가 어느 edge인가?

피해야 할 임시 해결 race가 사라질 때까지 임의의 #1 delay 추가
timeunit과 clock 주기가 바뀌면 관찰 위치도 달라져 문제를 다시 만든다.

한 문장 Interface는 신호를 묶고, modport는 역할을, clocking block은 timing을 정의한다.

LEARNING STAGE



5단계

UVM 개요와 전체 구조

transaction 흐름 · component 역할 · agent · checking · coordination

UVM 전체 구조와 transaction의 이동

무엇을 보낼지와 pin에서 어떻게 실행할지를 분리한다



핵심 질문 누가 만들고, 어디에 존재하며, 무엇을 생성·전달·변환하는가?
누가 pin을 다루고, 누가 correctness를 판단하는가?

정확한 재현과 넓은 탐색을 함께 사용한다

DIRECTED

addr=8'h10 · data=8'h55

특정 기능 · 경계값 · bug 재현

한계: 생각한 경우만 시험하기 쉬움

CONSTRAINED-RANDOM

addr inside {[8'h10:8'h1F]}

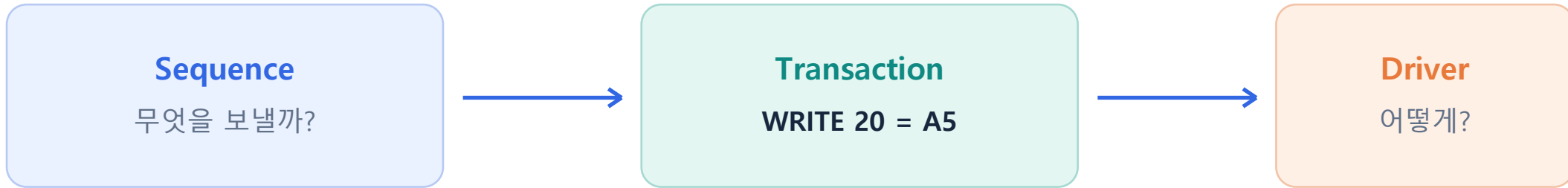
합법 공간 안에서 다양한 조합 탐색

필수: coverage로 실제 탐색 범위 확인

판별 기준 randomize() 사용 여부보다 값 공간을 하나로 고정했는지, 넓게 탐색하는지가 중요하다.

실전 Directed로 핵심과 재현 · constrained-random으로 조합 탐색

의미와 protocol timing을 분리해 재사용한다



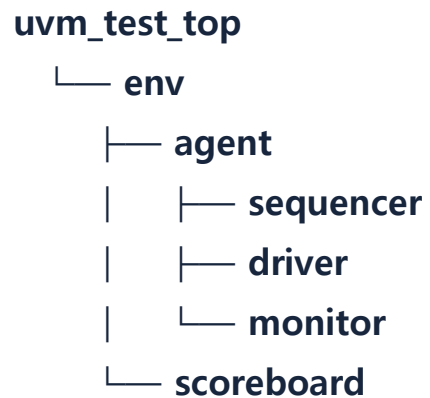
Transaction level 0x20 주소에 0xA5를 쓴다.

Pin level valid/addr/data 구동 → ready 대기 → handshake 완료

REUSE timing이 바뀌면 driver를 중심으로 수정하고 sequence의 검증 의도는 유지한다.

고정 구성 요소와 실행 중 생성되는 대상을 구분한다

UVM COMPONENT HIERARCHY



HIERARCHY 밖

Sequence 시나리오 실행 기간

Transaction 전달·처리 기간

UVM class이지만

component hierarchy에는 없음

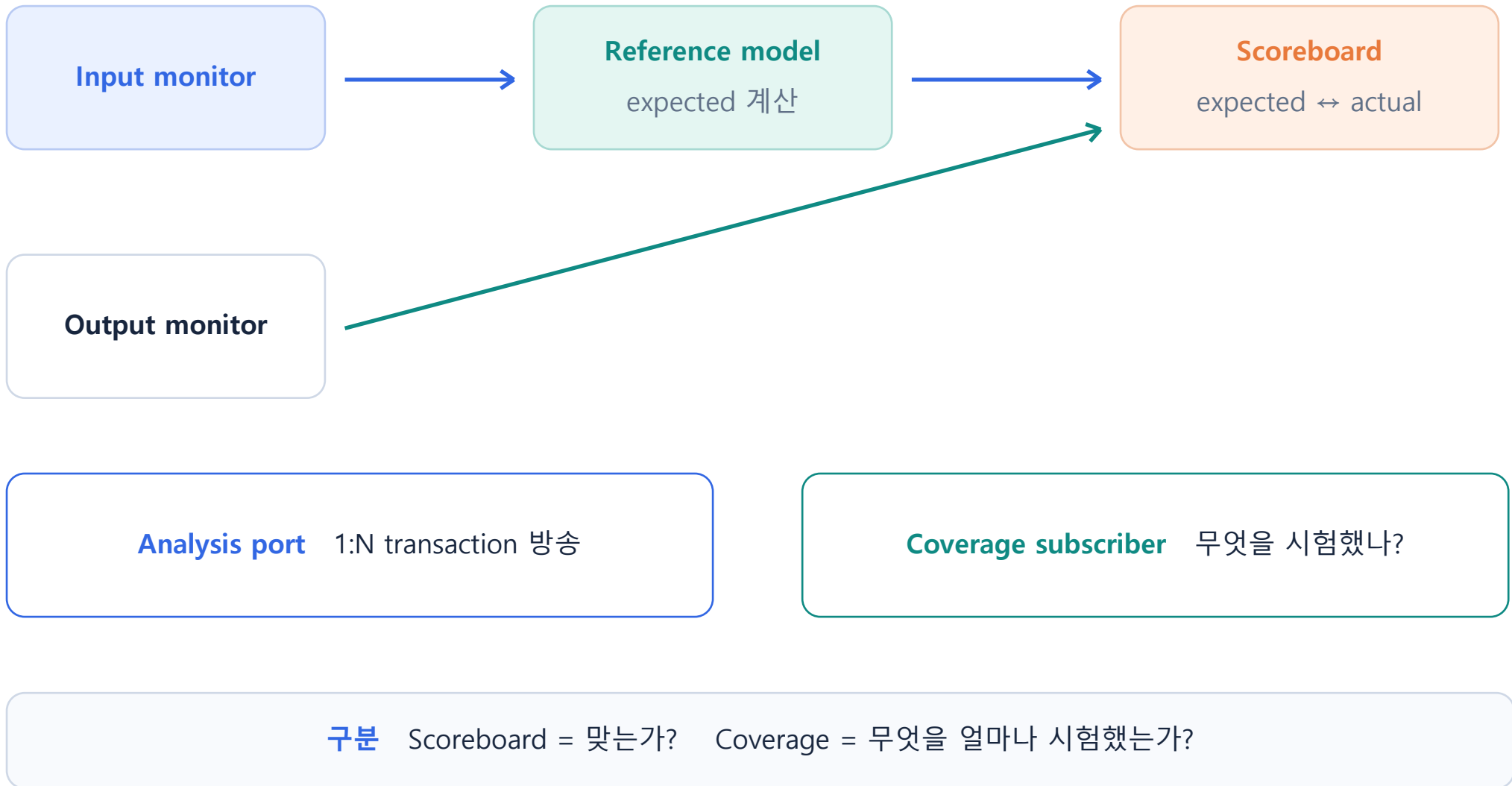
주의 test가 sequence를 시작해도 sequence가 test의 hierarchy 자식이 되는 것은 아니다.

Active는 구동과 관찰, passive는 관찰만 수행한다

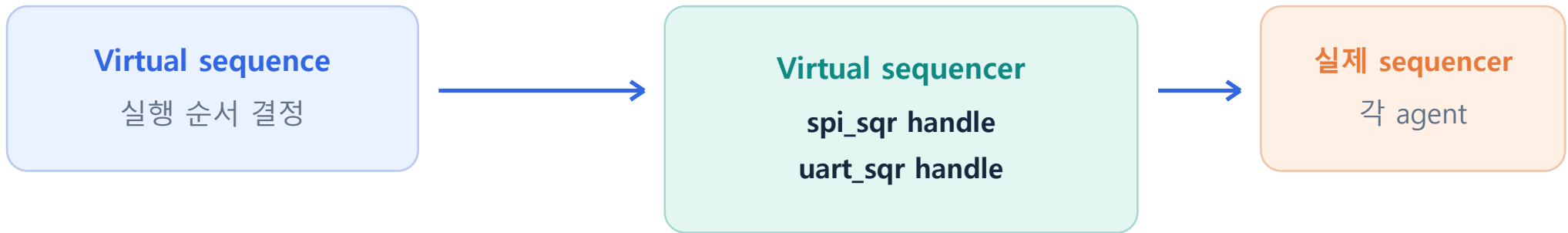


왜 active에도 monitor? driver의 의도가 아니라 interface에서 실제로 성립한 전송을 확인하기 위해서다.

관찰 사실, 예상값, 정답 판정을 분리한다



Virtual sequence는 지휘하고 virtual sequencer는 위치를 제공한다

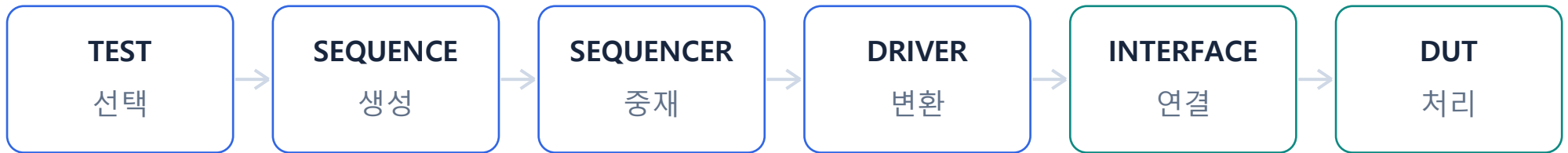


실제 구동 SPI sequence → SPI sequencer → SPI driver → SPI pin

핵심 virtual sequencer는 sequence 묶음이 아니며 일반적으로 자기 driver가 없다.

말로 순서는 virtual sequence · handle은 virtual sequencer · pin timing은 protocol driver

Transaction은 내려가며 signal이 되고, 올라오며 다시 transaction이 된다



자극 경로

관찰 경로



책임 경계 Sequence는 pin을 모르고 · monitor는 driver 의도를 모르고 · scoreboard는 pin을 직접 보지 않는다.

5단계 역할 점검

혼동했던 표현을 정확한 문장으로 바꾼다

1

SEQUENCE

UVM class지만 component hierarchy에는 없다.

2

SEQUENCER

sequence가 만든 transaction 전달을 중재한다.

3

MONITOR

실제 interface를 관찰해 새 transaction을 복원한다.

4

ANALYSIS

판단하지 않고 여러 subscriber에 방송한다.

5

VIRTUAL SEQ

여러 protocol sequence의 실행 순서를 지휘한다.

6

VIRTUAL SQR

실제 sequencer handle을 보관하며 pin을 구동하지 않는다.

다음 단계 uvm_object와 uvm_component를 hierarchy, phase, 수명, 생성 방식으로 분류한다.

LEARNING STAGE

6단계

uvm_object와 uvm_component

class 계열 · hierarchy · parent · phase · 생성 방식

전달되는 object와 구조를 이루는 component

이름이 같아 보여도 hierarchy, parent, 실행 방식이 다르다

ITEM / SEQUENCE

sequence item · sequence

component hierarchy 없음

phase 자동 호출 없음

UVM COMPONENT

test · env · agent · driver · monitor

이름 + parent로 hierarchy 구성

UVM phase에 참여

판별 질문 데이터나 시나리오인가? 검증환경에서 자리를 가진 지속 component인가?

핵심 Sequence는 UVM class지만 component hierarchy 밖에 있다.

상속도에서는 sequence도 sequence item의 자손이다

OBJECT 계열

```

uvm_object
├── uvm_transaction
│   └── uvm_sequence_item
│       ├── bus_item
│       └── uvm_sequence_base
│           └── uvm_sequence
    
```

COMPONENT 계열

```

uvm_object
├── uvm_report_object
│   └── uvm_component
│       └── test / env / driver / ...
    
```

component도 uvm_object의 후손이다.

구분 같은 component 계열이라는 사실이 서로 간의 class 상속을 뜻하지는 않는다.

extends와 parent는 서로 다른 관계를 표현한다

CLASS 상속

```
class bus_driver extends  
  uvm_driver #(bus_item);  
설계도 사이의 관계
```

COMPONENT 배치

```
create("driver", this)  
agent 객체 → driver 객체  
생성된 instance 사이의 관계
```

instance path `uvm_test_top.env.agent.driver`

driver class가 agent class를 extends한다는 뜻은 아니다.

Constructor와 factory 검색 context를 구분한다

OBJECT

```
new(string name = "bus_item")
bus_item::type_id::create("item")
create() parent = 선택적 factory context
component hierarchy 자식 등록은 아님
```

COMPONENT

```
new(string name, uvm_component parent)
bus_driver::type_id::create("driver", this)
this = 현재 parent component 객체
```

name은? "driver"는 instance 이름 · bus_driver는 class type
component의 parent=null은 UVM root 아래 배치를 뜻한다.

만들어진 후 무엇이 자동 실행되는지 확인한다

TRANSACTION / SEQUENCE

item: 필요할 때 생성해 전달·처리

sequence: 생성 후 start() 필요

start() → **body()**

COMPONENT

환경 구성 중 생성 후 지속

UVM이 hierarchy의 phase 호출

build → **connect** → **run**

100 transactions 보통 item 객체 여러 개를 driver component 하나가 차례로 처리
같은 item handle 재사용은 다른 수신자가 보관하면 값 변경 위험이 있다.

모두 hierarchy에 있지만 수행하는 일은 다르다

TEST

시나리오와 환경 선택

AGENT

한 protocol 묶음

DRIVER

item → pin 동작

SCOREBOARD

expected / actual 비교

ENV

agent와 검사 요소 구성

SEQUENCER

item 요청 중재

MONITOR

pin 관찰 → item

SUBSCRIBER

transaction 구독·처리

주의 Sequencer는 sequence들의 집합이 아니라 transaction 전달을 중재하는 component다.

6단계 분류 점검

계열 · hierarchy · 실행 방식 · parent를 함께 말한다

1

bus_item

object · item 데이터 · hierarchy 없음

2

write_sequence

object · start() 후 body() 실행

3

bus_sequencer

component · item 전달 중재

4

bus_driver

component · run_phase()에서 pin 구동

5

bus_scoreboard

component · transaction 비교

6

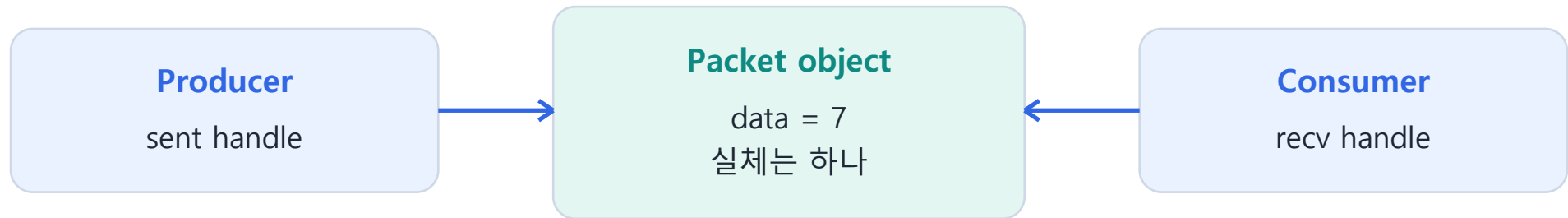
coverage subscriber

component · 측정, 보통 최종 판정은 아님

다음 단계 Factory 등록, override, utility macro의 실제 역할을 배운다.

전달했다고 복사된 것은 아니다

Mailbox의 handle 공유는 7단계 copy/clone으로 이어진다



mbx.put(sent) → mbx.get(recv)

공유 전달

sent.data=9 → recv.data=9
두 handle이 같은 object를 참조

독립 snapshot

새 object + 필요한 내부 object 복사
수신자가 보는 값은 별도로 유지

구분

Mailbox는 전달과 대기를 제공한다. Object의 복사 정책은 별도로 정한다.

LEARNING STAGE



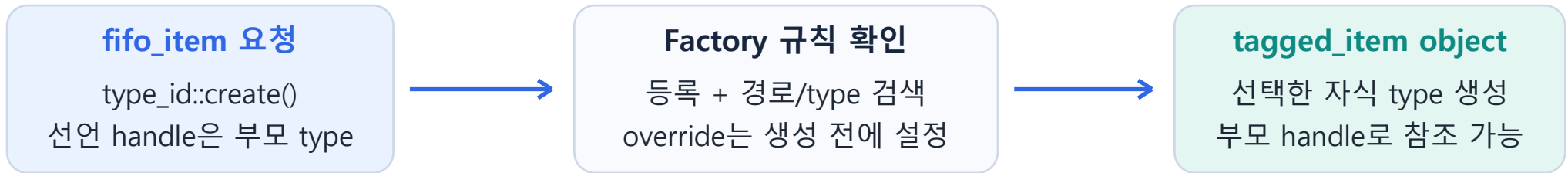
7단계

Factory와 utility macro

create / new · override · field 자동화 · copy / compare / clone

Factory는 생성할 type을 선택한다

이미 만들어진 object를 바꾸지 않는다. override는 다음 생성 요청에 적용된다.



생성 순서 / 두 class는 factory에 등록된 상태

```

fifo_item a, b, c;
a = fifo_item::type_id::create("a");
// tagged_item으로 type override 설정
b = fifo_item::type_id::create("b");
c = new("c");
  
```

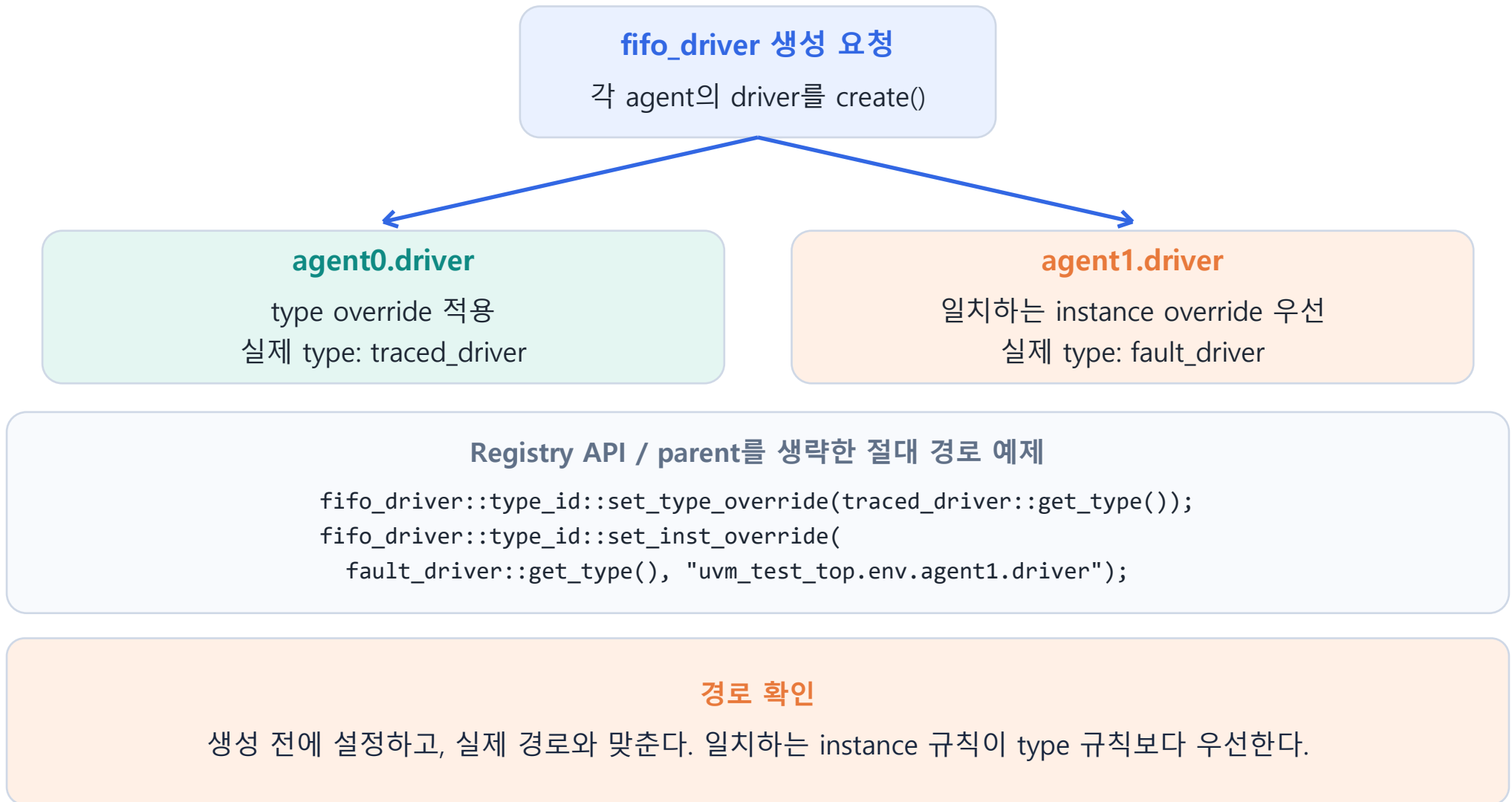
handle	실제 object type	생성 시점 / 방식
a	fifo_item	override 설정 전
b	tagged_item	설정 뒤 create()
c	fifo_item	직접 new()

기억할 점

a는 그대로다. b의 handle type은 fifo_item이고, 실제 object type은 tagged_item이다.

Type는 넓게, instance는 경로로 선택

같은 fifo_driver 생성 요청도 요청 경로에 따라 다른 자식 type을 받을 수 있다.



등록과 field 자동화는 별개의 기능

매크로를 전처리하는 순간 transaction object가 생기는 것은 아니다.

macro	factory / type 기능	사용자 field 자동화
uvm_object_utils(T)	유지	포함하지 않음
uvm_object_utils_begin / end	유지	uvm_field_*로 지정

component에는 uvm_component_utils 계열을 사용한다.

field별로 작업 참여 여부를 선택

```
`uvm_object_utils_begin(fifo_item)
  `uvm_field_int(data, UVM_DEFAULT)
  `uvm_field_int(debug_id, UVM_DEFAULT | UVM_NOCOMPARE)
`uvm_object_utils_end
```

NOCOMPARE

debug_id는 비교에서 제외한다. copy와 print는 별도 제외 flag가 없으면 참여한다.

값 복사와 handle 공유를 구분한다

목적지는 copy() 호출 전에 유효한 object여야 한다. 아래는 scalar data의 예다.

a.copy(b)

20을 복사한 뒤 b.data를 30으로 변경
a.data = 20 / b.data = 30

a = b

두 handle이 같은 object를 가리킴
b.data = 30이면 a.data도 30

super.do_compare()는 부모 class가 맡은 field를 비교한다.

비교 함수가 같은 함수인지 검사하는 연산이 아니다.

부모 field

addr 비교 결과 = 0



0 && 1 → 0

전체 비교 결과: 다름



자식 field

data 비교 결과 = 1

\$cast는 type 확인과 handle 획득이다. object를 복제하지 않는다.

비교 결과

부모 비교와 자식 비교가 모두 참이어야 한다. 부모 비교가 0이면 data가 같아도 전체는 0이다.

출력과 문자열 반환은 다르다

print / sprint는 printer 형식을 사용하고, convert2string은 직접 정한 문자열을 반환한다.

print()

printer 형식으로
화면에 출력

sprint()

printer 형식의
문자열을 반환

convert2string()

직접 작성한
문자열을 반환

do_print()는 print / sprint에 쓰는 field 정보를 printer에 제공한다.

문자열을 화면에 보이려면 출력 함수에 전달

```
string s1, s2;
item.print();           // 바로 출력
s1 = item.sprint();     // 문자열 저장
s2 = item.convert2string();
`uvm_info("ITEM", s2, UVM_LOW) // 반환값을 로그로 출력
```

반환과 출력

convert2string() 호출만으로는 로그가 생기지 않는다. 사용자 문자열 구현도 직접 작성한다.

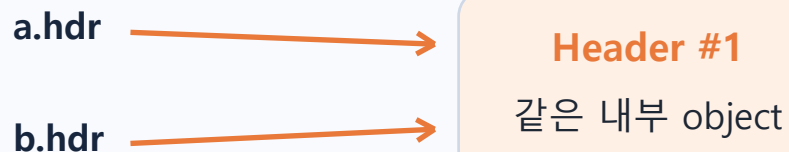
바깥 object와 내부 object를 따로 본다

복사한 `hdr.id`를 바꿀 때 원본에도 보이는지, 연결 그림으로 판단한다.

clone() = 새 바깥 object + copy()

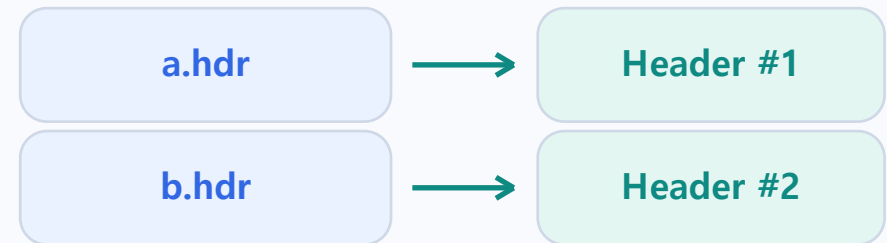
내부 handle의 공유 여부는 field 복사 정책과 구현으로 결정

UVM_REFERENCE



b.hdr.id = 9 → a에서도 9

UVM_DEEP



b.hdr.id = 9 → a의 id 유지

clone 반환 type은 `uvm_object` · 구체 handle에 담을 때 `$cast`

`$cast(b, a.clone());` // 실제 코드에서는 실패 여부를 확인

clone 주의

기본 clone의 `virtual create()`는 `type_id::create()`와 다른 method다.
clone을 factory override 재검색으로 이해하지 않는다.

7단계는 세 가지 구분으로 복습한다

먼저 왼쪽 질문에 답한 뒤 오른쪽 답안을 확인한다. 전체 class 작성과 simulator 실행은 아직 미평가다.

구분	핵심	확인할 지점
create / new	factory 규칙 / 직접 생성	override 설정 시점과 경로
utils / field	type 등록 / field 작업 자동화	작업별 제외 flag
copy / clone	내용 복사 / 새 바깥 object + 복사	내부 handle의 공유 여부

확인 문제

- ① override 뒤 create / new의 type?
- ② 부모 비교 0, data 같음 → 결과?
- ③ REFERENCE 내부 수정 → 원본 영향?

짧은 답안

- ① tagged_item / fifo_item
- ② 0: 부모와 자식 조건을 함께 만족해야 함
- ③ 있음: 같은 내부 object를 공유

PDF 보충

Parameterized class는 *_param_utils 계열을 사용한다. type 기반 등록·검색으로 확인한다.

LEARNING STAGE



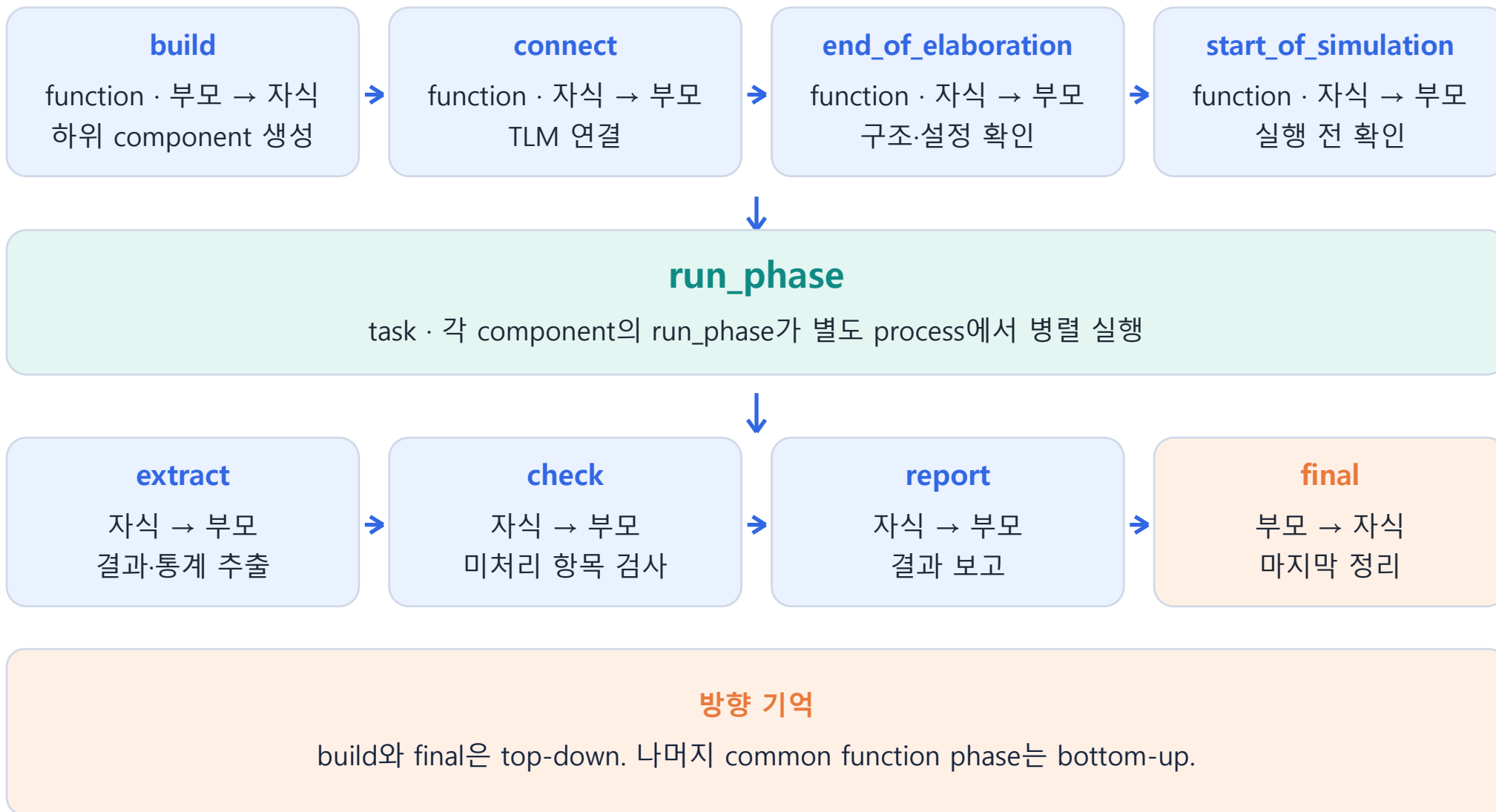
8단계

Phase와 objection

생성 → 연결 → 병렬 실행 → 실제 완료 확인 → 검사와 보고

만들고, 연결하고, 실행한 뒤 확인한다

phase 사이의 순서와 같은 phase 안의 hierarchy 방문 방향을 구분한다

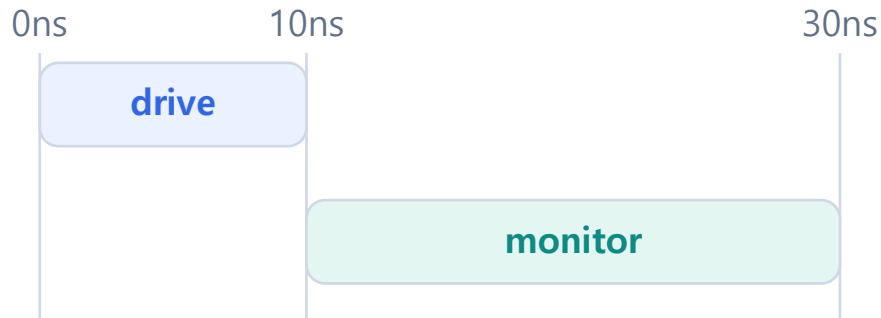


task라고 자동으로 동시에 실행되지는 않는다

function은 시간을 소비할 수 없고, task는 delay와 event를 기다릴 수 있다

일반 호출 · 순차 실행

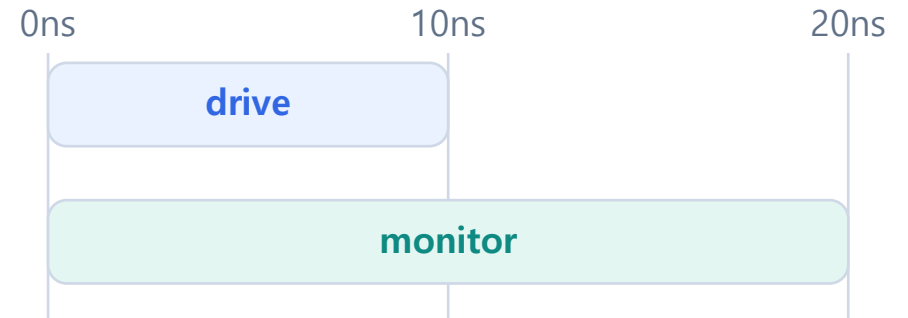
```
drive(); // 10ns 후 반환
monitor(); // 그 뒤 20ns 대기
// 전체 30ns
```



$$10 + 20 = 30ns$$

fork ... join · 병렬 실행

```
fork
  drive();
  monitor();
join // 두 process 종료까지 20ns
```



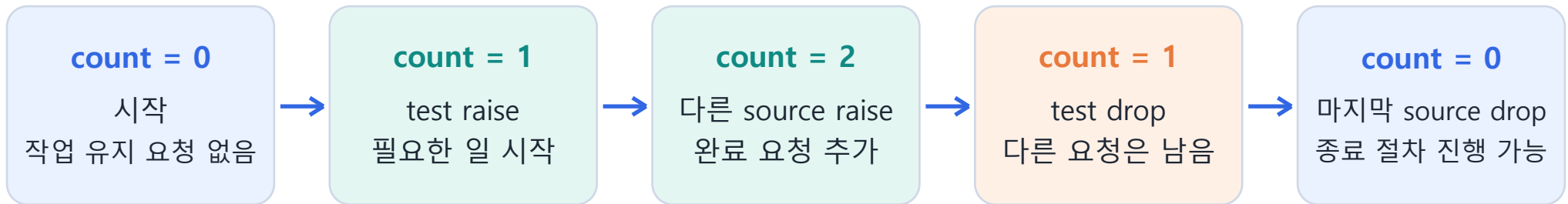
$$\max(10, 20) = 20ns$$

UVM의 병렬 실행

scheduler가 component별 run_phase process를 만든다. task keyword 자체가 병렬화를 하지는 않는다.

아직 할 일이 있으면 올리고, 끝나면 내려놓는다

raise/drop은 count를 관리한다. task가 실행 중이라는 사실만으로 phase가 유지되지는 않는다



driver forever / monitor forever

phase가 끝나면 그 phase에 속한 남은 process는 정리된다.

raise 없음

필요한 작업 전에
조기 종료 가능

drop 누락

count가 계속 남아
정상 종료 불가

너무 이른 drop

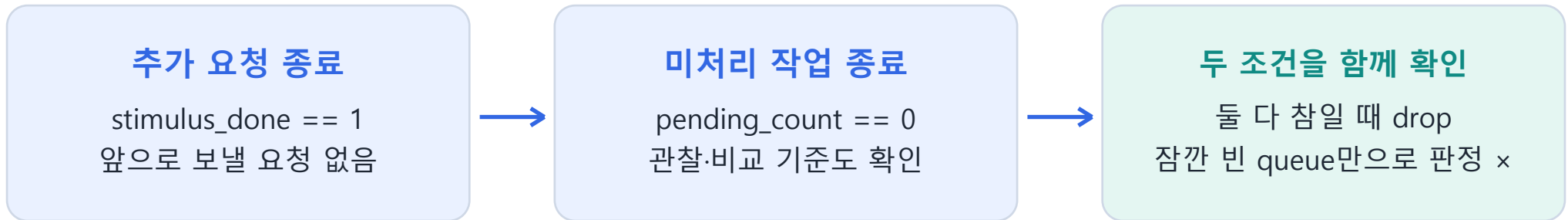
마지막 출력·비교를
놓칠 수 있음

종료의 의미

drop 한 번이 즉시 \$finish라는 뜻은 아니다. 남은 objection과 drain 등의 종료 절차 후 cleanup phase로 간다.

요청 전송 종료와 전체 검증 완료는 다를 수 있다

앞으로 올 요청이 없고, 이미 보낸 요청의 응답·비교도 모두 끝났는지 함께 확인한다



CODE / 개념 예제

```

phase.raise_objection(this);
send_requests();
wait (stimulus_done && scb.pending_count == 0);
phase.drop_objection(this);
  
```

if · 지금 한 번 확인

나중에 참이 돼도 자동으로 다시 실행되지 않음

wait · 참이 될 때까지 대기

이미 참이면 즉시 통과하고 다음 줄을 실행

완료 기준 설계

stimulus_done과 pending_count는 개념 예시다. 실제 DUT의 in-flight 관찰과 scoreboard의 대기 상태까지 정의한다.

test가 맡을 수도, component가 나눠 맡을 수도 있다

분산 관리도 가능하다. 각 component가 무엇을 완료로 판단하는지 연결해야 한다

A · Test가 전체 수명 관리

test raise



driver

요청 구동

monitor

출력 관찰

scoreboard

응답 비교



test drop

시나리오 종료 + 마지막 관찰·비교 완료

B · 각 component가 raise / drop

driver

100ns drop
미래 요청 없음

monitor

130ns drop
마지막
출력 관찰

scoreboard

135ns drop
더 올 입력 없음
비교 완료



모든 source의 count가 0

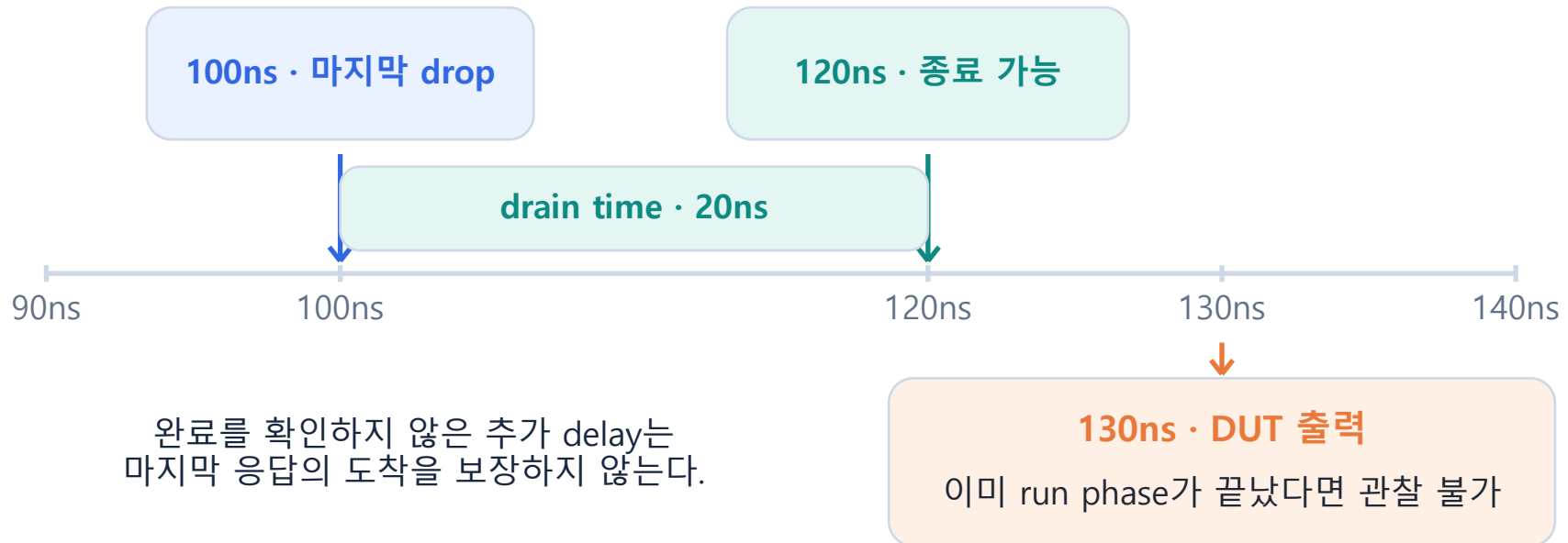
다른 지연이 없다면
135ns 이후 종료 가능

순환 대기 주의

monitor가 phase 종료를 기다린 뒤 drop하면, 자신의 objection이 그 종료를 막는다. 독립적인 완료 신호가 필요하다.

20ns를 더 기다려도, 130ns 출력은 놓칠 수 있다

마지막 drop = 100ns · drain = 20ns · 다른 objection이나 추가 callback 지연 없음



CODE / 개념 예제

```
objection = phase.get_objection();
objection.set_drain_time(this, 20ns);
```

사용 원칙

실제 완료를 확인한 뒤 drop하고 drain은 보조 여유로 둔다.
drain 중 새 objection이 올라가면 종료 절차를 다시 평가한다.

확장 phasing은 위치와 목적부터 알아둔다

아래는 자료 보충이다. 상세 구현과 확인 문제는 후속 학습에서 다룬다



run_phase와 기본 runtime schedule은 병행

runtime schedule 내부의 단계는 위 순서로 진행하며, 같은 phase의 component들은 병렬 실행한다.

Domain / phase jump

block별 phase 동기화 범위를 관리
jump 전후의 transaction-reset 상태 정리 필요

Sequence의 자동 objection

유효한 starting phase가 있어야 동작
forever sequence는 정상 drop에 도달하지 못할 수 있음

상세 내용

domain, callback, trace와 자동 objection의 API는 MD에 정리했다. sequence 실행 흐름은 9단계에서 이어간다.

시간, process, 그리고 완료 조건을 구분한다

설명하고 짧은 코드로 예측한 내용이다. UVM simulator에서 직접 실행한 예제는 아직 없다.

Q1 · 10ns task와 20ns task

일반 호출로 차례로 실행할 때와
fork ... join으로 실행할 때 전체 시간은?

Q2 · pending_count == 0

아직 뒤에 보낼 요청이 남아 있다면
지금 drop해서 검증 완료로 판단해도 될까?

Q3 · monitor가 objection을 올린 상태

phase가 끝난 뒤 drop하겠다고 기다리면
왜 종료가 진행되지 않을까?

짧은 답안

- ① 순차 30ns / fork 20ns ② 아직 drop 불가
- ③ 자신의 objection이 phase 종료를 막아 순환 대기가 된다.

LEARNING STAGE



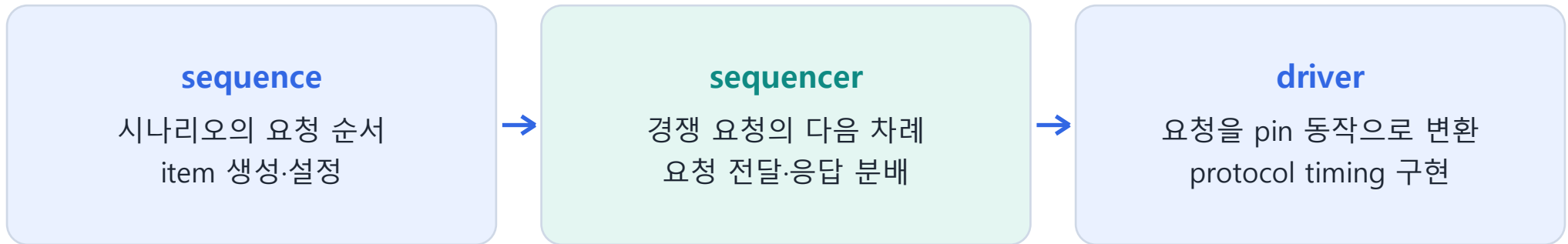
9단계

Sequence와 driver handshake

시나리오 실행 → 요청 중재 → 핀 구동 → 완료 통지

요청의 순서와 실제 신호 구동을 나눈다

Sequence는 object, sequencer와 driver는 component다



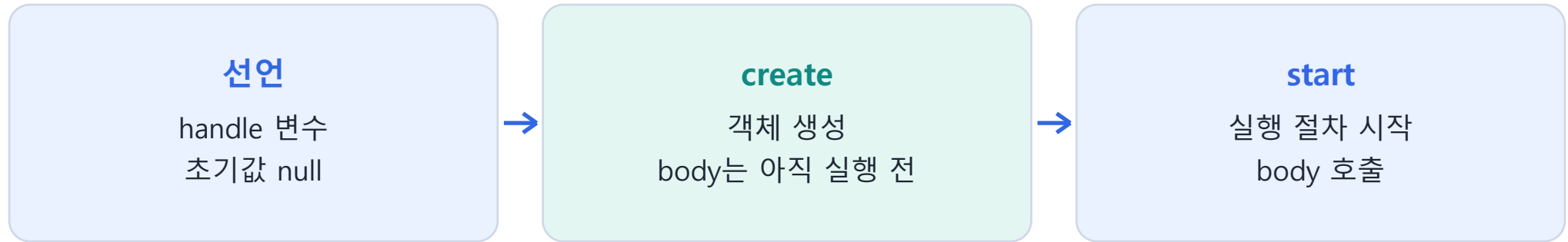
sequence item

요청 한 개의 데이터: wr_en / rd_en / data
객체 필드에 값을 넣는 것과 DUT pin을 바꾸는 것은 다르다.

책 연결

02.05 Sequences · 생성·실행·전달을 구분한다. 실제 port 연결은 11단계에서 설명한다.

객체를 만들었다고 body가 실행되지는 않는다



CODE / 개념 예제

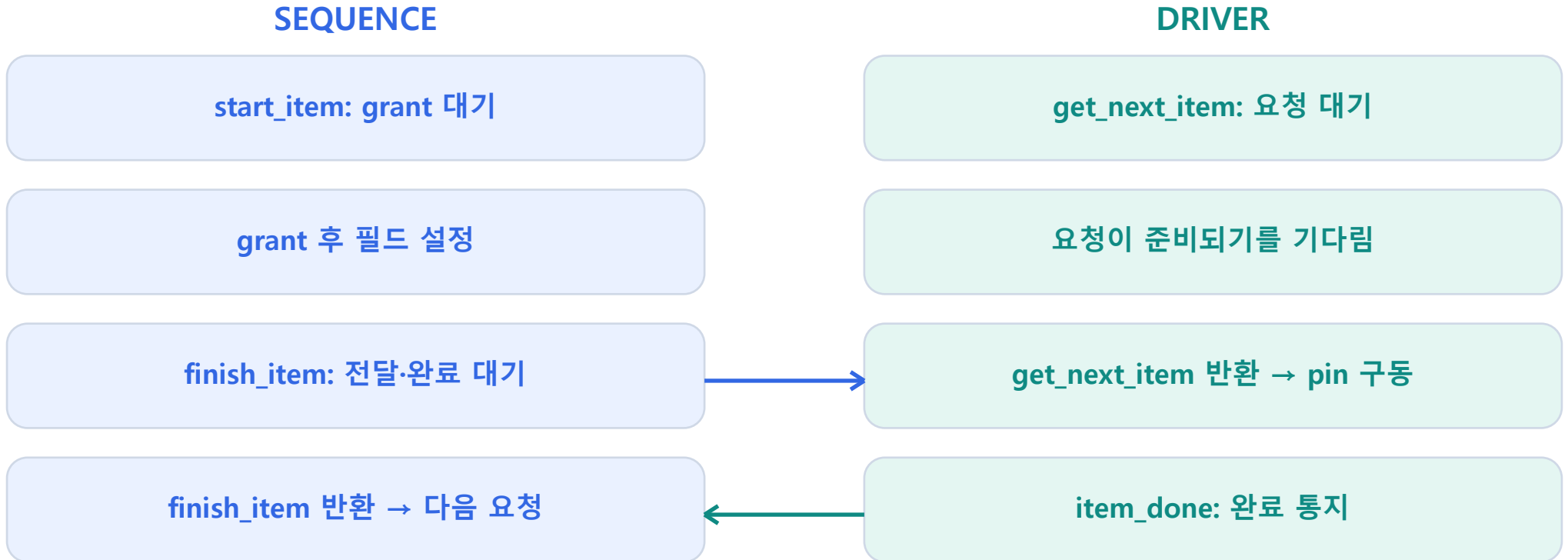
```
fifo_write_seq seq;
seq = fifo_write_seq::type_id::create("seq");
seq.count = 3;
seq.start(env.agent.sqr);
// start는 sequence가 종료될 때 반환한다.
```

Object의 실행

Sequence는 component phase로 자동 실행되지 않는다. 직접 start는 자동 randomization도 하지 않는다.

호출한 시점과 반환하는 시점을 구분한다

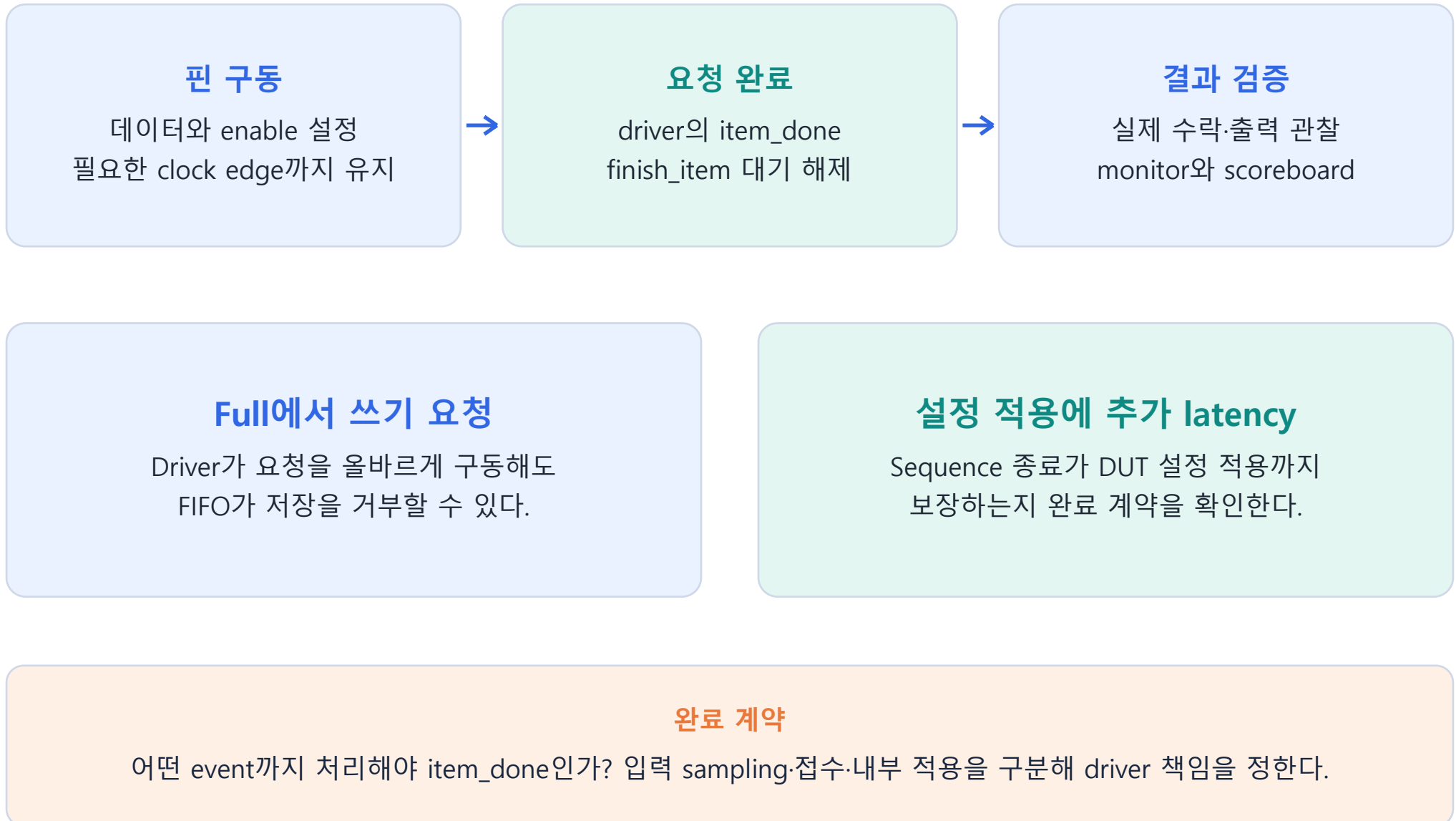
Driver는 `get_next_item`을 먼저 호출해 기다릴 수 있다



대기 위치

item_done이 없으면 finish_item의 완료 대기에서 멈춘다. finish_item 다음 로그는 완료 통지 후 실행된다.

구동 완료와 DUT 수락·정확성은 다르다



get은 수신 과정에서 handshake를 완료한다

EXPLICIT DONE

```
get_next_item(req);  
drive_one_item(req);  
item_done();
```

명시적 item_done 시점에
finish_item이 완료된다.

GET

```
get(req);  
drive_one_item(req);  
// item_done 추가 없음
```

요청 수신 때 handshake 완료
이후 pin 구동 완료는 별도다.

첫 FIFO driver

get_next_item + item_done으로 구동 완료와 handshake를 맞춘다. get 뒤의 실제 결과는 별도 통지로 관리한다.

완료 통지만으로 응답 객체가 생기지는 않는다

request

sequence → driver
동작을 요청



response

driver → sequence
필요한 결과를 반환

SEQUENCE

```
finish_item(req);
get_response(rsp);
// 응답이 없으면 대기
```

DRIVER

```
rsp = fifo_item::type_id::create("rsp");
rsp.set_id_info(req);
rsp.data = read_data;
seq_item_port.item_done(rsp);
```

선택적 응답

결과가 필요 없는 쓰기는 item_done만 사용해도 된다. 응답을 보내지 않으면 get_response도 호출하지 않는다.

시나리오 설정과 요청 데이터는 다른 객체다

seq.randomize()

요청 개수 count 등
시나리오의 rand 필드를 결정

req.randomize()

wr_en / rd_en / data 등
요청 한 개의 rand 필드를 결정

CODE / 개념 예제

```
seq.count = 3; // 직접 지정도 가능
seq.start(sqr);
// body: start_item(req);
//      req.randomize() with { wr_en == 1; rd_en == 0; };
//      finish_item(req);
```

시간 대기 금지

start_item 반환과 finish_item 호출 사이에는 필드 설정·randomization만 한다. Clock 대기는 driver에 둔다.

새 객체를 만들면 이전 비반복 상태는 이어지지 않는다



CODE / 개념 예제

```

rand_obj = fifo_item::type_id::create("rand_obj");
repeat (10) begin
  req = fifo_item::type_id::create("req");
  start_item(req); // 이후 randomize 실패 검사 필요
  if (!rand_obj.randomize()) `uvm_fatal("RAND", "failed")
  req.copy(rand_obj); finish_item(req);
end
  
```

책 179쪽

copy가 필요한 필드를 복사하도록 구현되어야 한다. randc의 비반복은 한 순환 안에서 보장한다.

한 sequence의 요청 순서를 뒤집지 않는다

한 sequence만 실행

A 쓰기 → B 쓰기 → 읽기
앞 finish_item 이후 다음 요청 시작

다른 sequence도 경쟁

A 쓰기 → 다른 요청 → B 쓰기
A와 B 사이에 요청이 끼어들 수 있음

담당	결정하는 내용
sequence	자기 시나리오 내부 요청 순서
sequencer	여러 sequence 사이 다음 item의 차례
driver	실제 pin 구동과 clock timing

중재의 범위

Arbitration은 경쟁 요청의 선택이다. 같은 sequence의 B를 A보다 먼저 보내도록 재배열하는 기능은 아니다.

다른 driver도 동작 의존성이 있으면 순서가 필요하다

SEQUENTIAL

```
write_seq.start(sqr, this);
read_seq.start(sqr, this);
// 쓰기 종료 후 읽기 시작
```

PARALLEL

```
fork
    write_seq.start(sqr, this);
    read_seq.start(sqr, this);
join // 둘 다 종료할 때 반환
```

Nested sequence

다른 sequence를 하위 시나리오로 조합
start는 종료까지 호출자를 기다리게 함

Virtual sequence

여러 sequencer의 하위 시나리오 조율
설정 적용 완료 후 데이터 시작 등

실행 순서

Driver가 다르다는 이유만으로 병렬 실행이 항상 맞지는 않다. DUT의 의존성과 완료 조건을 먼저 정의한다.

설정하는 시점과 실제 실행하는 phase를 구분한다

명시적 실행

Test가 객체를 create
원하는 task에서 start 호출

Default sequence

Build에서 타입.phase 설정
지정 phase 시작 시 sequencer가 생성.실행

CODE / 개념 예제

```
uvm_config_db#(uvm_object_wrapper)::set(  
    this, "env.agent.sqr.main_phase", "default_sequence",  
    fifo_write_seq::type_id::get()  
); // build_phase에서 설정만 수행
```

Objection

Default 설정만으로 phase 수명이 보장되지는 않는다. 자동 objection에는 유효한 starting phase가 필요하다.

요청 전달, 완료, 응답을 구분해서 설명한다

Q1 · `get_next_item` 이후 `item_done` 누락

Sequence는 어디서 기다리며 다음 요청은 어떻게 되는가?

Q2 · 같은 `sequencer`에서 여러 `sequence` 실행

A와 B의 순서를 뒤집는가, 다른 요청이 사이에 들어오는가?

Q3 · 응답 없이 `get_response` 호출

완료 통지와 응답 객체 수신은 같은 동작인가?

짧은 답안

- ① `finish_item` 완료 대기 ② 다른 요청이 사이에 들어올 수 있음
- ③ 별개이므로 응답이 없으면 `get_response`에서 대기

LEARNING STAGE

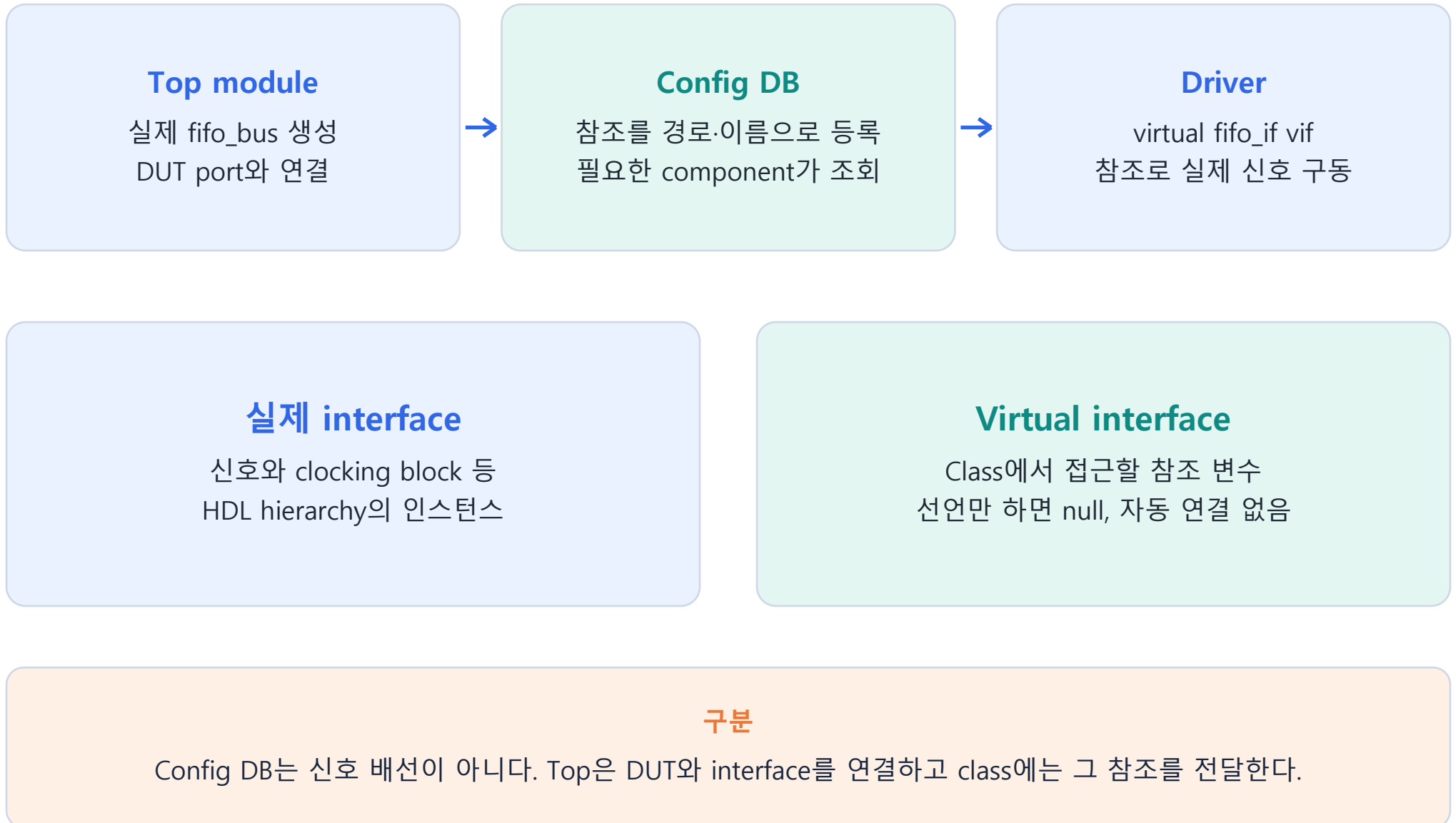


10단계

config DB와 virtual interface

실제 interface → 참조 등록 → 조회 → class에서 신호 접근

Virtual interface는 실제 interface를 가리킨다



Top이 등록하고 driver가 build에서 가져온다

TOP

```
fifo_if fifo_bus(clk);
initial begin
    uvm_config_db#(virtual fifo_if)::set(
        null, "uvm_test_top.env.agent.drv", "vif", fifo_bus);
    run_test();
end
```

DRIVER / BUILD_PHASE

```
virtual fifo_if vif;
if (!uvm_config_db#(virtual fifo_if)::get(this, "", "vif", vif))
    `uvm_fatal("NO_VIF", "setting not found")
if (vif == null) `uvm_fatal("NULL_VIF", "null reference")
```

조회와 참조

get 성공 여부와 가져온 참조의 null 여부를 각각 검사한다.

Handle 변수 이름과 component 이름은 다르다

CODE / 개념 예제

```
driver_h = fifo_driver::type_id::create("fifo_drv", this);
// driver_h: handle 변수 이름
// fifo_drv: UVM component 인스턴스 이름
```

context = this

현재 env의 full name
uvm_test_top.env



inst_name

상대 대상 경로
agent.fifo_drv



합쳐진 대상

uvm_test_top.env
.agent.fifo_drv

경로 확인

get_full_name()으로 실제 경로를 확인한다. agent 대상 설정이 drv까지 자동으로 확장되지는 않는다.

타입, 이름, 대상 경로를 맞춰 조회한다

확인 기준	등록	조회 실패 예
타입	int	string으로 get
설정 이름	vif	fifo_vif로 get
대상 경로	env.agent.drv	env.agent.mon에서 get
Interface 타입	virtual fifo_if	다른 parameter/modport 타입

agent.* 범위의 설정

하위 drv / mon / sqr가 각각 get해야 값을 저장한다. 너무 넓은 wildcard는 interface 참조를 잘못 공유할 수 있다.

명시적 조회

설정 범위에 포함된다고 component 변수가 자동 갱신되지는 않는다.

Build 중에는 상위 context의 설정이 우선한다

Test build: 200

context = uvm_test_top
대상 env.agent.drv

Env build: 100

context = uvm_test_top.env
대상 agent.drv

Driver가 가져오는 값: 200

Env가 나중에 등록해도 build 중 상위 test 설정이 우선한다.

조건	일반 set의 규칙
같은 우선순위	나중 등록 우선
Build 이후	기본 우선순위 적용, 나중 등록 우선

주의

구체적인 경로 문자열이 항상 이기는 것은 아니다. DB의 정수 설정 변경은 이미 get한 정수 변수를 바꾸지 않는다.

여러 설정을 agent 단위 객체로 묶는다

CODE / 개념 예제

```
class fifo_config extends uvm_object;
  `uvm_object_utils(fifo_config)
  int timeout_cycles = 100;
  bit checks_enable = 1;
  virtual fifo_if vif;
  // constructor는 MD 참고
endclass
```

생성 → 설정 → 전달

Test가 cfg 생성.필드 설정
config DB로 agent에 전달
Agent는 get 후 자식에게 전달
cfg.vif도 유효한 참조를 넣어야 한다.

TEST SET / AGENT GET

```
uvm_config_db#(fifo_config)::set(this, "env.agent", "cfg", cfg);
uvm_config_db#(fifo_config)::get(this, "", "cfg", cfg);
```

객체 공유

Config DB는 객체를 자동 clone하지 않는다. 기본적으로 같은 객체 handle을 전달한다.

내용을 수정하는 것과 새 객체 등록은 다르다

처음: 객체 A 공유

Test cfg / Agent cfg / DB

모두 객체 A를 가리킴

A의 필드 수정

Agent가 A의 필드를 읽으면
변경값을 볼 수 있다.

새 객체 B 등록 후

Test cfg / DB → 객체 B

Agent cfg는 여전히 객체 A

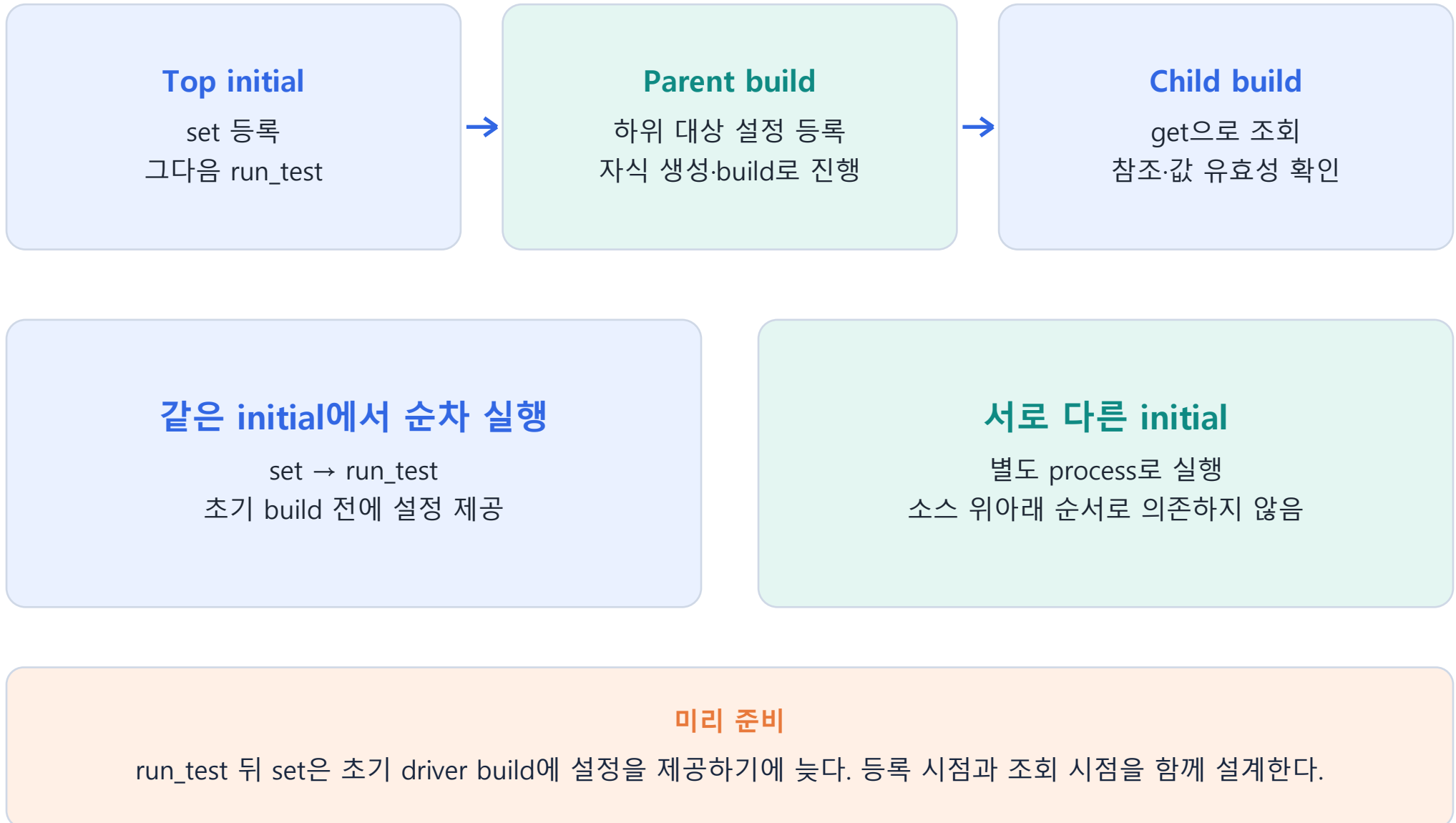
B의 필드 수정

A를 가리키는 Agent에는 영향 없음
새 handle은 다시 get해야 함

정수 복사와도 구분

cfg.timeout_cycles를 별도 정수 변수에 미리 복사했다면 공유 객체를 수정해도 그 정수는 자동 갱신되지 않는다.

설정이 필요한 build 전에 set을 완료한다



Config DB는 component 계층에 맞춘 설정 접근이다

비교	resource DB	config DB
범위	scope 문자열 직접 지정	context + 상대 경로
등록·조회	set / read_by_name 등	set / get
주 활용	공유 resource	component별 설정 전달

같은 resource 기반

서로 독립된 DB로 보지 않는다. Resource DB도 scope를 가지며 항상 전역에만 쓰는 것은 아니다.

기본 선택

Virtual interface와 configuration object를 component 경로로 전달할 때는 config DB를 기본으로 사용한다.

조회 실패와 null 참조는 별도로 확인한다

1 · 실제 component 경로

get_full_name / create 이름 / parent

2 · 등록·조회 조건

field_name / type / scope / parameter·modport

3 · 시점·우선순위

get 전에 set했는가? 상위 설정이 덮는가?

4 · 유효한 참조

get 성공이어도 vif와 cfg는 null 검사

로그 보충

+UVM_CONFIG_DB_TRACE로 조회·등록 추적, print_topology로 구조 확인. 실제 trace 실행은 13단계 실습에서 한다.

실제 interface와 class의 참조를 연결해 설명한다

Q1 · create("fifo_drv", this)

Handle 이름 driver_h와 인스턴스 이름 중 경로에 들어가는 것은?

Q2 · DB에 새 cfg 객체 등록

이전에 get한 agent의 cfg가 자동으로 새 객체를 가리킬까?

Q3 · get 성공, vif는 null

이 참조로 실제 pin을 구동할 수 있을까?

짧은 답안

- ① fifo_drv ② 다시 get해야 새 handle을 받음 ③ 참조 대상이 없어 접근 불가
다음 채팅: 11단계 TLM·analysis 이론 / FIFO 실습은 이후 재개

1~10단계 빠른 찾기 · 1~5단계

PDF는 그림으로 복습 / MD는 자세한 이론·코드·대화 확인 기록

단계	핵심 주제	시작 페이지
1	객체지향 문법	2
2	데이터 구조와 Process 통신	11
3	Randomization과 Constraint	20
4	Interface와 Simulation Timing	29
5	UVM 개요와 전체 구조	38

자료: 회사 PDF 02장 / UVM 1.2 Reference · 상세 출처는 단계별 MD

현재: 10단계 기본 이론·코드 해석 완료 / UVM simulator와 FIFO 실습 미실행

다음: 11단계 TLM-analysis 이론 → 12~14단계 → Callback·RAL / 세미나 준비 우선

1~10단계 빠른 찾기 · 6~10단계

PDF는 그림으로 복습 / MD는 자세한 이론·코드·대화 확인 기록

단계	핵심 주제	시작 페이지
6	uvm_object와 uvm_component	48
7	Factory와 utility macro	57
8	Phase와 objection	65
9	Sequence와 driver handshake	74
10	config DB와 virtual interface	87

자료: 회사 PDF 02장 / UVM 1.2 Reference · 상세 출처는 단계별 MD

현재: 10단계 기본 이론·코드 해석 완료 / UVM simulator와 FIFO 실습 미실행

다음: 11단계 TLM-analysis 이론 → 12~14단계 → Callback·RAL / 세미나 준비 우선